hydra_satsuma: Certified Structure Dispatch with a satsuma-iter+kissat Fall-Through

Greg Sidebottom

Research note — logic repo (https://github.com/gsidebottom/logic), SAT track, 2026-08-12. Solver description for the `hydra_satsuma` *backend of* `sat`*: the certified* `hydra` *structure-dispatch pipeline (Cook-shape prover, XOR/GF(2) stage) whose fall-through engine is satsuma-iter+kissat, winner of the SAT Competition 2026 main track. Performance sections will be completed after the three-way apples-to-apples run (*`hydra` */* `satsuma` */* `hydra_satsuma`*) on the official 2026 benchmark set on this machine.*

Abstract

`hydra_satsuma` is a structure-dispatch SAT solver: before any CDCL search, it tests the input for algebraic structure that admits a *polynomial-size certificate*, and only when no structure applies does it hand the formula to a general-purpose engine. Two structural stages run in order: (1) a *Cook-shape prover* that recognizes five cardinality-contradiction families (pigeonhole and its generalizations) and emits the Cook-1976 extension-variable refutation as a VeriPB proof — families on which resolution, and hence any DRAT-based route without new variables, is exponential; (2) an *XOR/GF(2) stage* that recovers parity constraints from their clausal encodings and runs Gaussian elimination, certifying parity refutations in VeriPB after Gocht–Nordström. The fall-through engine is satsuma-iter+kissat (Anders et al.), the SAT Competition 2026 main-track winner, run unmodified in a container: satsuma detects symmetries with dejavu and emits a binary substitution-redundancy (SR) proof of its breaking clauses, kissat appends its clausal refutation to the same file, and `dsr-trim` verifies the *composed* proof against the exact CNF handed over — so symmetry-broken UNSAT results remain checker-certified. Every UNSAT path thus produces a machine-checked certificate in a format on the SAT Competition 2026 checker menu (VeriPB) or in the winner's own SR chain; SAT results carry the model as witness, projected to the original variables.

1 Pipeline

The backend runs the following stages in order; the first stage to reach a verdict ends the run. Stage budgets and caps are given with each stage.

1.1 Cook-shape prover

A syntactic detector (inputs up to 10^6 clauses) recognizes five structured UNSAT families, reading the exact clause layout rather than solving:

- **PHP-n-m** — the canonical pigeonhole formula, n pigeons, m holes, $n > m$;
- **RoundRobin** — sports-scheduling infeasibility, a per-pair/per-day two-level pigeonhole;
- **Embedded pigeonhole** — P disjoint at-least-one clauses over S complete slot-mutex cliques with $P > S$, the generalization covering e.g. MVRoundRobin;
- **Clique-coloring** — a graph asserted to contain a K-clique yet be C-colorable with $K > C$, proved by composing the clique-membership and coloring layers;
- **Mutilated chessboard** — bipartite perfect-matching infeasibility with imbalanced sides, proved by the two-sided cardinality argument.

On a match the verdict is UNSAT *by construction*, in microseconds to milliseconds, and — when a proof is requested — the module emits the corresponding pseudo-Boolean refutation in VeriPB format. For PHP the proof is exactly Cook's 1976 construction: fresh variables $Q_{ij} = P_{ij} \vee (P_{im} \wedge P_{nj})$ encode a pigeonhole instance with one fewer pigeon and hole; the reduction iterates down to a trivially refutable base. Each layer is polynomial, rendered as VeriPB `red` steps with the definition as witness. This matters because Haken's 1985 lower bound makes PHP exponential for resolution: a solver-emitted DRAT proof (which cannot introduce variables the solver never used) is out of reach at scale, while the extension-variable route is short, and VeriPB is one of the three checkers on the SAT Competition 2026 menu (§2.2).

1.2 XOR/GF(2) stage

For inputs up to 5×10^6 clauses, the stage recovers XOR constraints from their CNF encodings (complete canonical encodings only; the certifier re-derives the recovery strictly, so no trusted parsing) and runs GF(2) Gaussian elimination. Three outcomes:

- **Inconsistent system** (by unit propagation or by GE): the formula is UNSAT. When a proof is requested the stage emits a parity refutation in VeriPB after Gocht–Nordström (2021) — the certificate names the (typically small) subset of recovered XORs whose GF(2) sum is $1 = 0$. Parity families are another resolution-exponential class certified polynomially here.
- **Pure-XOR satisfiable**: GE solves the system outright; the model is the certificate.
- **Partial simplification**: GE forces some variables and simplifies the clause set, yielding an equisatisfiable *residual*. What happens next depends on *certified mode* (§1.3).

1.3 Certified mode

The `--proof` flag is the certified-mode switch for the whole hydra family. The subtlety is the partial-simplification case: an engine refutation of the *residual* does not certify the *original* formula — the GE step connecting them is real reasoning that the downstream proof does not contain. Therefore:

- **Certified mode** (`--proof` present; the benchmark harness always passes it): the GE simplification is *discarded* and the engine solves the original formula. Full end-to-end certification beats the simplification — a trade measured to be nearly free (§3).
- **Uncertified mode** (no `--proof`): the engine solves the residual. Verdicts remain sound (GE derives only logical consequences; SAT models are reconstructed by overwriting GE-forced variables), but a residual-path UNSAT is reported explicitly as uncertified for the original.

Emitting a proof *of the GE step itself* (deriving the forced units and simplified clauses via extension variables, in DSR or VeriPB) would close this gap and is well-understood machinery; it is deliberately deferred because measurement shows nothing is currently lost (§3).

1.4 Fall-through: satsuma-iter+kissat

When no structural stage decides the instance, `hydra_satsuma` hands it to the SAT Competition 2026 main-track winner, built unmodified from the official competition tarball (Anders et al.) and run in an Ubuntu 24.04 container:

1. `satsuma fix <cnf> --bsr --proof-file proof.out --out-file sb.cnf` — symmetry detection with dejavu (graph automorphisms of the colored literal–clause incidence graph), structure-based symmetry-breaking clauses, and a *binary SR proof* justifying each breaking clause with its symmetry as substitution witness;
2. `kissat sb.cnf proof.out` — the CDCL engine solves the broken formula and *appends* its clausal refutation to the same proof file;
3. on UNSAT, `dsr-trim -f <cnf> proof.out` verifies the *composed* proof — satsuma's SR prefix plus kissat's refutation — against the exact CNF handed over.

The composition is the point: symmetry-breaking clauses are not consequences of the formula (they are satisfiability-preserving, not equivalence-preserving), so a bare engine refutation of the broken formula certifies nothing about the input. SR's substitution witnesses make the breaking steps checkable, and because kissat's clause additions are valid SR steps, one checker validates the whole file. SAT models over satsuma's augmented variable space are projected back to the original variables (sound: breaking clauses only remove models).

Operationally the container runs are bounded end to end:

- **Solve phase**: in-container `timeout` at the remaining budget minus 2 s, so the container always self-terminates and is never orphaned by the host-side watchdog. The verdict and solve time are recorded *before* any verification.
- **Verify phase**: a second bounded container run (`--satsuma-verify-secs`, default equal to the solve timeout, matching the harness's proof-check budget convention for the LRAT chain; 0 = unlimited). A verification timeout downgrades the row to sound-but-unchecked; the verdict is never lost.
- **Memory**: optional per-container hard cap (`--satsuma-mem-gb`); an instance exceeding it fails alone.
- **Size guard**: instances with $\geq 10^6$ variables or $\geq 2.5 \times 10^7$ clauses skip satsuma (its literal graph on such inputs is several GB) and run kissat directly on the raw formula; an UNSAT proof remains dsr-trim-checkable, since kissat's steps are the suffix of every composed proof.

The sibling backends complete the family: `hydra` (same structural stages, CaDiCaL fall-through with native LRAT checked by `cake_lpr`) and `satsuma` (the winner bare, no structural stages — the baseline arm for measuring what the dispatch adds).

2 Certificates by path

Path	Verdict	Certificate	Checker
Cook shape	UNSAT	Cook-1976 extension proof (VeriPB)	`veripb`
XOR inconsistent	UNSAT	GN21 parity proof (VeriPB)	`veripb`
Pure-XOR solvable	SAT	model witness	—
satsuma+kissat	UNSAT	composed binary SR	`dsr-trim`
satsuma+kissat	SAT	model (projected)	—
GE residual (uncert. mode)	UNSAT	none for the original	—

Verification of the SR path happens *in-solver* (the second container phase), and the harness credits it from the solver's `dsr-trim VERIFIED UNSAT` marker rather than re-checking — binary SR is not readable by the LRAT/GRAT/VeriPB checkers, and routing it to one of them would produce a spurious rejection.

2.1 Checker cost: hinted vs. unhinted

The two verification chains do structurally different work. `cake_lpr` consumes LRAT, where every step carries unit-propagation hints computed by CaDiCaL at solve time; checking is a near-linear replay. `dsr-trim` consumes unhinted DSR — like `drat-trim`, it must find every propagation itself and discharge each SR step's redundancy goals, i.e. it performs the elaboration that the LRAT path amortized into the solve. A same-CNF measurement on this machine (instance `b20`, ISCAS circuit, UNSAT):

Chain	Proof	Size	Check time
CaDiCaL native LRAT	hinted	40 MB	`cake_lpr` 1.55 s
satsuma+kissat DSR	unhinted	20 MB	`dsr-trim` 7.8 s

SR's compensation is succinctness (the same instance's SR proof is half the LRAT's size; Codel–Avigad–Heule report SR proofs at 0.4% of the line count of their RAT translations) and expressiveness (the symmetry steps have no DRAT analogue without extension variables). Memory behaves oppositely: `dsr-trim` is a compact C program (no out-of-memory events across concurrent checking in our runs), while `cake_lpr`'s CakeML runtime requires a pre-sized heap that scales with the proof and is the component our harness bounds with a dedicated memory cap and a concurrency semaphore.

2.2 Competition-format compliance

SAT Competition 2026 accepts three proof checkers in the main track: DPR (`dpr-trim`), GRAT (`gratgen`), and VeriPB (`veripb`). The Cook and GN21 certificates are VeriPB proofs and verify under the same `veripb` the competition runs; the SR chain is the 2026 winner's own. One gap separates `hydra_satsuma` from literal entry shape: the rules have a solver declare a *single* checker and write one `proof.out`, whereas this backend emits per-path formats. Unification is mechanical future work — either translate the engine's clausal refutations into VeriPB (`rup`/`del` steps map directly; RAT additions become `red` with witness) and declare VeriPB for everything, or render the Cook/GN21 constructions in DRAT-with-extension-variables and declare the clausal chain.

3 Measured design decisions

Two measurements on the official 2026 main-track set (400 instances with duplicates; this machine) fixed design choices in advance of the full performance run:

- **GE simplification is worth ≈ nothing on this set.** Every instance on which the partial-simplification path fires (twelve, all ISCAS-class circuits) recovers exactly one XOR and forces exactly one variable — on formulas of 2×10^3 to 3.4×10^5 variables. Solve-only A/B (original vs. residual, same engine): `b20` 7.43 s vs. 7.40 s, `s38417` 1.66 s vs. 1.68 s, `c7552` 0.75 s vs. 0.77 s — within noise. Certified mode's "ignore GE, solve the original" therefore costs nothing measurable, and building the GE-step certifier (§1.3) is not currently justified by performance.
- **Where the XOR stage does pay, it is already certified.** In the prior full `hydra` run on this set, eleven instances were decided outright by the parity stage in ∼10 ms each, all eleven with verified GN21 certificates; twenty-seven more fell to the Cook prover with verified VeriPB proofs.

4 Results (pending)

To be completed: the three-way comparison on the official SAT Competition 2026 main-track set (400 instances including duplicates, 5000 s per instance, certified mode, ten parallel workers, all arms on this machine with the satsuma arms in Linux/Docker):

- `hydra` — structural stages + CaDiCaL/LRAT (baseline: 318 solved in the initial run of the deduplicated set);
- `satsuma` — the 2026 winner bare, as shipped;
- `hydra_satsuma` — structural stages + the winner.

The comparison will report solved counts, per-instance and cactus times, and the certification ledger (verified / unchecked / failed per chain), measuring both what structure dispatch adds to the winner and what the winner's engine adds to `hydra`.

Acknowledgments

Claude (Fable 5, Anthropic) assisted with the implementation, measurement, and drafting throughout. satsuma and dejavu are by Markus Anders et al.; kissat by Armin Biere; `dsr-trim` and `lsr-check` by Cayden Codel; `cake_lpr` by Yong Kiam Tan, Marijn Heule, and Magnus Myreen; VeriPB by Stephan Gocht, Jakob Nordström, et al. The satsuma-iter+kissat components are built unmodified from the official SAT Competition 2026 submission archive and run as-is in a container.

References

[1] S. A. Cook. The complexity of theorem-proving procedures. *STOC*, 1971.

[2] S. A. Cook. A short proof of the pigeon hole principle using extended resolution. *SIGACT News*, 8(4):28–32, 1976.

[3] A. Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.

[4] S. Gocht and J. Nordström. Certifying parity reasoning efficiently using pseudo-Boolean proofs. *AAAI*, 2021.

[5] C. R. Codel, J. Avigad, and M. J. H. Heule. Verified substitution redundancy checking. *FMCAD*, 2024. https://www.cs.cmu.edu/~mheule/publications/SRcheck.pdf

[6] Y. K. Tan, M. J. H. Heule, and M. O. Myreen. `cake_lpr`: Verified propagation redundancy checking in CakeML. *TACAS*, 2021.

[7] F. Pollitt, M. Fleury, and A. Biere. Faster LRAT checking than solving with CaDiCaL. *SAT*, 2023.

[8] SAT Competition 2026: certificates and checkers. https://satcompetition.github.io/2026/output.html; winner submission archive https://satcompetition.github.io/2026/downloads/solvers/anders.tar.xz.

[9] M. Anders et al. Simplify, order, break, repeat (satsuma-iter). *SAT*, 2026, https://drops.dagstuhl.de/storage/00lipics/lipics-vol377-sat2026/. satsuma: https://github.com/markusa4/satsuma.